

针对稀疏量子态制备的最优线路研究

摘要

量子态制备问题是量子计算的基本问题，也是量子线路编译的关键部分。在一般量子态制备的算法中，生成的线路规模会随着量子比特数量呈指数增长。然而，许多实际相关的状态在标准基中较为稀疏，这种稀疏性在态制备问题中有着天然的优势。基于此，我们提出了一种多项式时间算法，该算法能够在不使用辅助比特的情况下针对给定的稀疏态设计对应的实现线路进行精确制备。我们数值验证了当非零相位数量等于比特数量时，相比于目前最优的稀疏态制备算法，当比特数量为 10 时，我们算法可以节省 19.73% 的线路长度；当比特数量为 15 时，我们算法可以节省 22.28% 的线路长度；当比特数量为 50 时，我们算法可以节省 16.86% 的线路长度。考虑 NISQ 设备的特定结构，我们通过增加线路局部性将原始全连接电路在保持逻辑等价的前提下优先重写为近邻交互，将电路高效映射到 Heavy-Hex 与 Square 等非全连接耦合图上，与目前最优方法相比，当量子比特数量为 10 时，Heavy-Hex 拓扑结构可以节约 19.70% 的线路长度，Square_25 可以节约 26.18% 的线路长度，最高可以节约 37.60% 的线路长度。同时，我们设计一个软件包，改软件包好上手，可以促进使用人员的算法探索，也便于没有量子线路基础的研究人员使用该算法。

关键词 量子计算；量子线路优化；量子态制备；稀疏态制备；多控 Toffoli 门

1 引言

量子计算自提出以来就表现出解决经典难解问题的强大潜力，其中一些著名的算法有区分平衡函数和常函数的 DJ 算法^[1]、量子无序搜索算法^[2]、量子大整数分解算法^[3]等。此外近些年，量子机器学习^[4]、量子哈密顿量模拟由于其相对经典算法的优越性^[5-8]都得到了广泛的研究和讨论。量子计算俨然已成为各个国家科研的新动力新方向，在 2024 年 3 月展开的十四届人大第二次会议上李强总理也提到“要开展量子技术新赛道”。

在这个 NISQ 时期，量子编译俨然成为了连通量子算法与量子设备的桥梁，能够将量子算法在量子设备上部署实现。而在量子编译中有一个关键的问题是量子态制备问题，即创建特定初始量子态以编码输入数据或问题实例的过程。量子态的制备对于现有量子算法的实现起着关键作用。通常，量子算法工作的起点就是将通用经典数据编码为量子态。然而，由于叠加态和纠缠态的特殊现象，量子编译过程的某些环节与经典计算中的对应任务存在显著差异，无法直接进行数据迁移。这就会造成量子态制备这一问题往往会产生较大的线路开销。

作为量子计算、量子线路编译的重要部分，量子态制备的复杂度远超经典计算的模拟过程^[9]。在经典计算中，通过设置 $O(n)$ 个量子门即可获得 $O(n)$ 大小存储单元的所有可能状态；而量子计算机的状态是基态的叠加态，其中部分状态只能通过极其复杂的电路才能实现^[10-11]。事实上，研究表明，仅使用单量子比特门和 CNOT 门的门库时， n 量子比特量子计算机的一般量子态制备只能通过规模为 $\Omega(2^n)$ ^[10-11] 的线路来制备。由于一般量子态制备的线路成本较高，且实际应用中往往呈现特殊结构，已经存在针对振幅由连续函数给出的状态、通过黑盒预言机获取振幅的状态等特殊量子态^[12-17]制备的研究成果。

然而，与一般态不同的是，现实中大多数的量子态都是只有极少数基态具有非零系数，我们将这类具有该特性的态称为稀疏量子态。并且出现了许多针对稀疏量子态制备的相关研究。在不使用辅助比特的情况下，Gidney 等^[18-19]提出的方法以 $O(|S|n)$ 的线路复杂度实现稀疏态制备，随后 Li^[20]又改进到 $O\left(\frac{|S|n}{\log n} + n\right)$ 。在使用两个辅助比特的情况下 Mao^[21]提出的方法使用 $O\left(\frac{|S|n}{\log n}\right)$ 的线路复杂度实现稀疏态

制备。然而，针对实际制备的线路数值进行比较，Gidney^[18]提出的相位缩减方法仍为目前最优方法。

基于此，本文针对稀疏量子态制备的线路长度进行研究，提出了一种多项式时间算法。与先前工作不同的是，本文以实际应用为出发，研究线路长度的实际数值优化。与目前数值最优的稀疏态制备方法^[18]相比，本文算法在比特数量和量子非零相位数量都为 10 时，我们算法可以节省 19.73% 的 *CNOT* 门；当比特数量和量子非零相位数量都为 15 时，我们算法可以节省 22.28% 的 *CNOT* 门。并且随着非零相位数量的增加，我们的算法会表现出更大的优势。当在比特数量为 10 非零相位数量为 50 时，我们算法可以节省 25.54% 的 *CNOT* 门。

本文结构如下，第二节介绍准备工作，第三节介绍我们的方法，第四节为数值实验评估和我们的工具包，第五章为结论和未来展望。值得注意的是，我们给出了一个稀疏态制备的案例，和我们软件包的使用方法。

2 准备工作

2.1 稀疏量子态

在包含 n 个量子比特的系统中，计算基态共有 2^n 个相互正交的向量 $|00\dots 00\rangle, |00\dots 01\rangle, \dots, |11\dots 11\rangle$ ，分别对应长度为 n 的二进制串。任意量子态 $|\psi\rangle$ 可写成这些基态的归一化线性组合：

$$|\psi\rangle = \sum_{c \in \{0,1\}^n} \alpha_c |c\rangle, \quad \sum_{c \in \{0,1\}^n} |\alpha_c|^2 = 1 \quad (1)$$

其中 α_c 称为振幅。若绝大多数 $\alpha_c = 0$ ，仅少量非零，则称该态为稀疏量子态。记非零振幅对应的二进制串集合为 $S \subset \{0,1\}^n$ ，则

$$|\psi\rangle = \sum_{c \in S} \alpha_c |c\rangle \quad (2)$$

当 $|S| \ll 2^n$ 时，只需存储集合 S 及其对应的非零振幅 α_c ，即可比完整记录 2^n 个系数更高效地描述该量子态。

2.2 量子门与量子线路

1. 单量子比特门

对任意单量子比特态 $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ ，单量子比特门由 2×2 的西矩阵 U 实现，满足

$$U^\dagger U = U U^\dagger = I_2,$$

其中 $U^\dagger$ 表示 U 的共轭转置， I_2 为 2 维单位矩阵。该门作用后的态为

$$|\psi'\rangle = U|\psi\rangle$$

一般地，单量子比特门可实现任意 $SU(2)$ 变换。

2. CNOT 门（受控非门）

CNOT 门作用于两量子比特，设第一个比特为控制比特，第二个比特为目标比特，其矩阵表示为

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

该门在计算基下的作用为

$$\text{CNOT}|c\rangle|t\rangle = |c\rangle|c \oplus t\rangle, \quad c, t \in \{0,1\}$$

在本文中，将选取单量子比特门与 *CNOT* 门作为基础门进行量子线路构建。

3. 多控 Toffoli 门

量子线路中的多控 *Toffoli* 门（Multi-controlled Toffoli，记作 MCT_k ）是经典可逆逻辑门 k -Toffoli 的量子推广。给定 $(k+1)$ 个量子比特，其中前 k 个为控制位 $|c_1 c_2 \dots c_k\rangle$ ，第 $(k+1)$ 个为目标位 $|t\rangle$ ， MCT_k 的作用定义为

$$MCT_k |c_1 \dots c_k\rangle |t\rangle = |c_1 \dots c_k\rangle |t \oplus (c_1 \cdot c_2 \dots c_k)\rangle \quad (3)$$

当且仅当所有控制位均为 $|1\rangle$ 时，目标位受 X 门作用；否则保持不变。

其矩阵形式为 $2^{k+1} \times 2^{k+1}$ 的单位矩阵, 仅对角元 $X_{2^{k+1}-2, 2^{k+1}-2} = X_{2^{k+1}-1, 2^{k+1}-1} = 0$, 其余对应位置为 Pauli- X 元素, 即

$$MCT_k = I^{\otimes(k+1)} - |1 \dots 1\rangle\langle 1 \dots 1| \otimes I + |1 \dots 1\rangle\langle 1 \dots 1| \otimes X \quad (4)$$

特别地, 当 $k = 2$ 时, MCT_2 即为标准三比特 Toffoli 门 (CCX)。在计算基下的酉作用亦可简洁地写成

$$MCT_k |c\rangle |t\rangle = |c\rangle |t \oplus \prod_{i=1}^k c_i\rangle, \quad c_i, t \in \{0, 1\} \quad (5)$$

后续线路构造以 MCT_k 作为核心门集之一; 借助 Gidney^[22] 的方案, 该门集可在 $O(\log |S|)$ 个 CNOT 门内实现。

4. 多控制 G 门

在本文的量子线路构造中, 我们需要借助一类特殊的单量子比特门

$$G(\alpha, \omega) = \begin{pmatrix} e^{i\alpha} \cos \omega & \sin \omega \\ \sin \omega & -e^{i\alpha} \cos \omega \end{pmatrix}, \quad (6)$$

该门作用在基态上满足 $G(\alpha, \omega)(\alpha|0\rangle + \beta|1\rangle) = e^{i\varphi}|0\rangle$ 其中 $G(\alpha, \omega)$ 可被高效地实现为多控门: 只需将其分解为两个单量子比特门与一个多控 Toffoli 门, 而多控 Toffoli 门又可进一步用线性数量的 CNOT 门实现^[23]。

5. 量子线路

量子线路是由量子门通过量子门连接而成的序列。若线路由门 $G_1, G_2, \dots, G_k$ 依次作用, 其整体酉矩阵为 $U_{\text{circuit}} = G_k G_{k-1} \dots G_1$ 为便于阅读, 线路图中允许使用多控操作。但在计算线路长度时, 由于现有的多 Qubit 量子门的保真度较低, 我们使用 CNOT 门的数量作为线路长度开销。

2.3 问题表述

给定集合 $S \subset \{0, 1\}^n$ 及非零系数 $\{\alpha_c\}_{c \in S}$ 所定义的量子态 $|\psi\rangle = \sum_{c \in S} \alpha_c |c\rangle$, 构造由基础量子门 $G_1, G_2, \dots, G_k$ 组成的线路 $C = G_k G_{k-1} \dots G_1$ 使得

$$C |0\rangle^{\otimes n} = |\psi\rangle \quad (7)$$

并要求尽可能减少门数 k 。由于基础门可逆, 若存在线路 C' 满足 $C' |\psi\rangle = |0\rangle^{\otimes n}$, 则取逆线路 $(C')^{-1}$ 即可实现正向制备; 故在后文中我们统一描述为“寻找将 $|\psi\rangle$ 映射到 $|0\rangle^{\otimes n}$ 的线路”。

3 方法

本文的核心思路为将 $|\psi\rangle$ 拆分为若干“非零相位缩减”的子问题, 通过特定受控操作实现两个非零相位合并, 最后迭代执行上述操作, 最终把所有非零振幅集中到 $|0 \dots 0\rangle$ 。因此, 降低每一次两个非零相位合并的门开销成为关键。若能恰当的减少实现两非零相位合并所需 G 门的控制位数量, 就能显著减少所需门数。后文将围绕这一思路展开: 先总体展示合并思路, 再通过特定算法体现如何选择控制位、受控位与实现门电路的构建, 最后通过对比实验, 展示该算法的有效性。总体思路实现如 Algorithm 1。

Algorithm 1 稀疏态制备

```

1: Input: 一个  $n$  量子比特量子态的经典描述  $\phi = \sum_{x \in S} c_x |x\rangle$ 
2: Output: 一个电路  $C$  使得  $C|0^n\rangle = \phi$ 。
3: 初始化一个空的量子电路  $C$ ;
4: while  $|S| > 1$  do
5:   使用 Algorithm 2 确定需合并的非零相位与多控 G 门的控制位  $control\_bit$ ;
6:   使用 Algorithm 3 确定多控 G 门的受控位  $dif$ ;
7:   使用 Algorithm 4 找到一个线路  $\hat{C}$  先实现的两个非零相位仅在受控位  $dif$  位上存在差异, 且  $control\_bit$  位均为 1, 再进一步实现在量子比特  $dif$  上应用一个由  $control\_bit$  中的量子比特控制的  $G$  门, 实现  $\hat{C}\phi = \sum_{x \in S'} c'_x |x\rangle$ , 其中  $|S'| = |S| - 1$ 
8:   更新状态  $\phi = \hat{C}\phi$ ;
9:   更新电路  $C = \hat{C} \circ C$ ;
10: end while
11: 在  $C$  中添加将  $\phi$  转换为  $|0^n\rangle$  所需的 NOT 门;
12: 反转  $C$  中的门并颠倒它们的顺序;
13: Return:  $C$ ;

```

3.1 控制位选择与预合并非零相位确立

根据 Algorithm 1 可知, 算法的核心步骤之一在于对两个非零相位用控制 $G(\alpha, \omega)(\alpha|0\rangle + \beta|1\rangle) = e^{i\varphi}|0\rangle$ 实现合并。在实际操作过程中, 存在控制 G 门操作意外导致其他非零相位的变化, 所以选取存在显著区别的 $control_bit$ 对于实现非零相位合并的意义重大。具体要求为当且仅当预合并非零相位的 $control_bit$ 全为 1 且其他非零相位不完全满足该条件时, 才允许实现多控 G 门操作。为实现该要求, 我们通过对每一个量子比特进行最大差异化的选择迭代, 快速确立控制比特与合并量子态。具体算法实现见 Algorithm 2。算法中的一个核心思想是, 由于多控 G 门的开销大, 我们利用 $CNOT$ 门进行处理, 使得后续的多控 G 门的控制位减少, 这是算法高效的关键所在。

Algorithm 2 多控 G 门控制比特选择与预合并非零相位确立

```

1: Input: 0-1 矩阵  $M \in \{0,1\}^{s \times n}$ , 其每一行对应一个计算基态  $|x_k\rangle$ 。
2: Output: 控制比特集合  $control\_bit$ ; 满足  $M_{r_1,q} = M_{r_2,q}$  的两行下标  $r_1, r_2$ ; 已施加的 CNOT 电路  $C$ 。
3: for  $j = 1$  至  $n$  do
4:    $w_j \leftarrow \sum_{k=1}^s M_{k,j}$  ▷ 计算第  $j$  列的汉明重量
5:   if  $w_j = 2$  或  $w_j = s - 2$  then
6:     选取满足  $M_{r,j} = M_{r',j}$  的两行  $r_1, r_2$  ▷ 即所选比特值相同
7:     return  $(j, r_1, r_2, C = \emptyset)$ 
8:   end if
9: end for
10:  $C \leftarrow \emptyset$  ▷ 初始 CNOT 门序列为空
11: while True do
12:   接受上次循环输出的新矩阵, 若第一次循环则接受原矩阵
13:   for  $i = 1$  至  $n$  do
14:     for  $j = i + 1$  至  $n$  do
15:       设  $w_{i \oplus j}$  表示列  $i$  与列  $j$  逐位异或后的重量
16:       设  $\Delta_{i,j}$  表示  $i$  列与  $j$  列异或前后, 最小重量的差值
17:         
$$\Delta_{i,j} = \min(w_i, w_j, s - w_i, s - w_j) - \min(w_{i \oplus j}, s - w_{i \oplus j})$$

18:       if  $\Delta_{i,j} > 1$  且  $\max(w_i, w_j, s - w_i, s - w_j) = \max(w_i, s - w_i)$  then
19:         将  $CNOT(i, j)$  追加到  $C$ , 表示  $i$  为受控位,  $j$  为控制位并更新第  $j$  列:
20:         
$$M_{k,i} \leftarrow M_{k,i} \oplus M_{k,j}, \quad \forall k \in [1..s]$$

21:       else if  $\Delta_{i,j} > 1$  且  $\max(w_i, w_j, s - w_i, s - w_j) = \max(w_j, s - w_j)$  then
22:         重复第一个判断操作, 但是替换  $i, j$  的位置
23:       end if
24:       if  $w_j = 2$  或  $w_j = s - 2$  或  $w_i = 2$  或  $w_i = s - 2$  then
25:         选取满足  $M_{r,q} = M_{r',q}$  的两行  $r_1, r_2$ , 其中  $q$  为  $i$  或  $j$ 
26:         return  $(control\_bit = q \cup control\_bit, r_1, r_2, C)$ 
27:       end if
28:     end for
29:   end for
30:   取满足  $\min_{1 \leq i \leq n} \min(w_i, s - w_i)$  的第  $i$  列对应的量子比特加入控制比特集合  $control\_bit$ 
31:   if  $w_i < s - w_i$  then 去除第  $i$  列中所有不为 1 的行生成新的矩阵
32:   else 去除第  $i$  列所有不为 0 的行生成新的矩阵
33:   end if
34: end while

```

Algorithm 2 的前 7 行首先在各列中查找汉明重量 (Hamming weight) 恰好为 2 或 $s - 2$ 的列。该策略的理论依据如下。

设待合并的两个计算基态在某一行 q 上取值相同 (0 或 1), 而在其余列上至少存在一位差异。若列 q 的汉明重量为 2 或 $s - 2$, 则意味着:

- (1) 当重量为 2 时, 仅有 2 行在该列取 1;
- (2) 当重量为 $s - 2$ 时, 仅有 2 行在该列取 0。

在这两种情况下, 列 q 天然地提供了一种区分性: 只有待合并的两行在列 q 上呈现相同的比特值 (0 或 1), 其余所有行在该列上均取相反值。此时, 若选取列 q 作为控制位, 并以其余差异位为受控位施加 CNOT 门, 即可在不影响其他基态的前提下, 消除除列 q 之外的所有差异。随后, 若仍需消除列 q 本身的差异,

可进一步以两非零相位在该列的共同值为 1 的位（若共同值为 0，可先施加 NOT）作为控制位，执行受控 G 门操作。由于列 q 已经提供了独占性的区分特征，该组合操作不会导致其他计算非零相位的变化，从而确保合并过程的安全性。

如果步骤一未能找到满足条件的列，算法进入迭代阶段（行 9–30）。此时，矩阵中不存在汉明重量为 2 或 $s - 2$ 的列，但目标依旧是构造具有最小汉明重量或其补集的列，以逼近上述理想区分特性。步骤 11–12 通过逐列异或并计算差值 $\Delta_{i,j}$ 来评估是否可用一次 CNOT 变换显著降低某列的有效重量。若 $\Delta_{i,j} > 1$ ，则以被替换列为受控位、未替换列为控制位施加 CNOT，并更新矩阵。一旦新生成的矩阵中出现重量为 2 或 $s - 2$ 的列，立即返回该列作为控制位。

若经过以上操作仍未出现理想列，则表明单个量子比特无法独立提供区分性。此时，算法退而求其次，选择当前矩阵中 $\min\{w_i, s - w_i\}$ 最小的列作为首个控制位。由于该列在全局上具有最小的不确定性，移除对应取值的反例行后，剩余子矩阵与原问题同构，可递归地重复上述流程，直至最终锁定仅包含两个待合并非零相位的子空间。

3.2 受控位选择

根据 Algorithm 2 我们已经成功选取了 2 个预合并非零相位、控制比特与新的矩阵，接下来我们需要进一步选定受控位。受控位选择的算法如 Algorithm 3。

Algorithm 3 选择多控 G 门差异位算法

```

1: Input: 算法 2 中加入 CNOT 门之后量子态对应的 0-1 矩阵  $M$ ，算法 2 中获取的确立的合并非零相位分别为  $r_1, r_2$ 
2: Output: 对应受控位的量子比特  $dif$ 
3:  $min\_H = \text{float}('inf')$ 
4: for 每列  $j$  do
5:   if  $M(r_1, j) \neq M(r_2, j)$  then
6:      $col\_values = [\text{row}[col\_idx] \text{ for } row \text{ in } matrix]$ 
7:      $H(j) = \min(col\_values, \text{len}(col\_idx) - \text{one\_count})$ 
8:     if  $H(j) < min\_H$  then
9:        $min\_H = H(j)$ 
10:       $dif = j$ 
11:    end if
12:  end if
13: end for
    return  $dif$ 

```

根据 Algorithm 3，可以选出两非零相位之中不相等的量子比特 dif 。算法任意选取两非零相位的差异位 dif 作为受控位，这为下节中进一步消除两非零相位差异与最后实现两非零相位合并作了准备。

3.3 门电路实现

根据 Algorithm 2 和 Algorithm 3，我们确定了控制位和受控位，在 Algorithm 4 中需要最终实现两非零相位的合并。

Algorithm 4 量子线路构建

```

1: Input: Algorithm 1 中构建的电路  $C$ , 合并非零相位  $r_1, r_2, dif, control\_bit$ 
2: Output: 量子态合并电路  $C$ 
3: if  $r_1[dif] \neq 1$  then
4:   在  $C$  中添加一个作用于  $dif$  的 NOT 门;
5: end if
6: for  $b$  在  $\{1, 2, \dots, n\} \setminus \{dif, control\_bit\}$  中 do
7:   if  $r_1[b] \neq r_2[b]$  then
8:     在  $C$  中添加一个以  $dif$  为控制位, 作用于  $b$  的 CNOT 门; ▷ 消除其他比特差异
9:   end if
10: end for
11: for  $b$  在  $control\_bit$  中 do
12:   if  $r_2[b] \neq 1$  then
13:     在  $C$  中添加一个作用于  $b$  的 NOT 门; ▷ 确保控制比特均为 1
14:   end if
15: end for
16: 以  $dif\_qubit$  为控制位,  $dif$  为受控位, 应用控制 G 门实现两非零相位合并

```

通过 4-8 行的操作, 以 dif 位为控制位、以两非零相位中取值不同的所有量子比特为受控位, 可将除 dif 外的全部差异一次性消除。随后, 为确保所有 $control_bit$ 均置 1, 对两非零相位中为 0 的量子比特施加 NOT 门。由 Algorithm 2 可知, 所选 $control_bit$ 具有最大差异特征; 以 $control_bit$ 为控制、以 dif 位为目标作用 G 门, 可保证仅对选定两非零相位实现合并, 而不会令其他非零相位发生变化。

最后, 按 Algorithm 1 的 while 循环逐步合并所有成对非零相位, 直至系统变为全零态。将该过程对应的电路整体逆序, 即可得到制备目标稀疏态所需的完整量子线路。

3.4 线路局部性的态制备算法

在本文提出的算法中, 受控旋转与条件判断逻辑不可避免地需要大量 CNOT 门来传递和累积相位信息; 然而当前 NISQ 设备的物理拓扑稀疏、耦合图直径大, 远程 CNOT 必须通过多次 SWAP 操作实现, 极易成为后续映射阶段的瓶颈。为此, 本文在原线路构建框架之上引入“局部优先”的合成策略, 增强线路构建算法在实际设备中的鲁棒性。

3.4.1 态制备线路局部性

在控制位确定算法的异或搜索阶段中, 我们引入“距离权重”作为遍历策略, 实现对近距离量子比特的优先搜索。

Algorithm 5 按距离排序节点对

```

1: Input: 物理量子比特拓扑结构
2: Output: 物理量子比特距离矩阵列表
3:  $n \leftarrow$  量子比特数
4: 节点对列表  $\leftarrow []$  ▷ 存储任意两个量子比特极其距离信息
5: for  $i \leftarrow 0$  到  $n - 2$  do
6:   for  $j \leftarrow i + 1$  到  $n - 1$  do
7:      $d \leftarrow D[i][j]$ 
8:     节点对列表.append( $[i, j, d]$ )
9:   end for
10: end for
11: 排序后列表  $\leftarrow$  按距离升序(节点对列表)
12: return 排序后列表

```

在 Algorithm 5 中, 我们获得了按距离排序的节点对。以此作为依据, 在 Algorithm 2 的遍历阶段中, 算法优先对近距离比特进行异或搜索, 降低 CNOT 门的构建成本。

3.4.2 多控 G 门控制位选择线路局部性

在多控 G 门的控制位选择过程中, 若只以汉明重量作为参考可能导致 SWAP 成本飙升, 进一步导致综合成本的增加, 因此需综合考虑控制位的减少与物理比特之间的距离。

Algorithm 6 多控 G 门控制位选择策略

```

1: Input: 0-1 逻辑矩阵 M、物理比特距离矩阵 D、已选控制位集合 C
2: Output: 最优控制位
3:  $v[] \leftarrow 0$  ▷ 控制位价值数组
4: for 候选列  $i = 0 \dots n$  do
5:   if  $i \in C$  or  $\omega_i \leq 1$  then ▷ 排除汉明重量为 1 或 s-1 的列与已选控制位对应列
6:     continue
7:   end if
8:    $\min\_dis = \min\{D[i][j] \mid j \in C\}$ 
9:    $v[i] = \alpha_1 \cdot \min\{\omega_i, s - \omega_i\} + \alpha_2 \cdot \min\_dis$  ▷ 汉明重量越小、距离越近越优
10: end for
11: 选取综合价值最优列  $i^* = \arg \max_{i \in N} v[i]$ 

```

Algorithm 6 在原算法的选择策略中引入权重机制, 对“控制位削减”与“距离”两项指标进行加权综合评估, 实现线路局部性的增加。

3.5 算法经典时间复杂度分析

在这里, 我们分析该系数量子态制备的算法经典时间复杂度。

算法 1 为稀疏态制备算法, 该算法遍历了所有的非零相位, 因此其经典时间复杂度为 $O(|S|)$, 并且每一次遍历调用了算法 2、算法 3、算法 4。

算法 2 为控制比特的选择算法, 该算法中计算初试汉明重量的经典时间复杂度为 $O(|S|n)$ 。后面使用 CNOT 门来缩减后面的多控 G 门的控制位, 这一部分的经典时间复杂度为 $O(|S|n \log n)$, 其中 $O(|S|n)$ 遍历计算汉明重量, $O(|S|n)$ 为层层叠加控制位时的控制位数量。

算法 3 为多控 G 门受控位的选择, 这里只需要 $O(n)$ 的时间复杂度。算法 4 为线路的合成, 同样是需要 $O(n)$ 的时间复杂度。

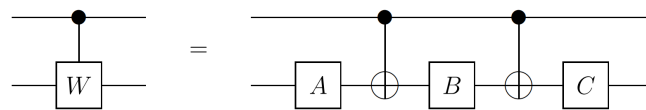
综合上面的分析, 本算法时间开销为 $O(|S|) * O(|S|n + |S|n + |S|n \log n + n + n)$, 简化为 $O(|S|^2 n \log n)$ 。

3.6 多控 G 门的分解

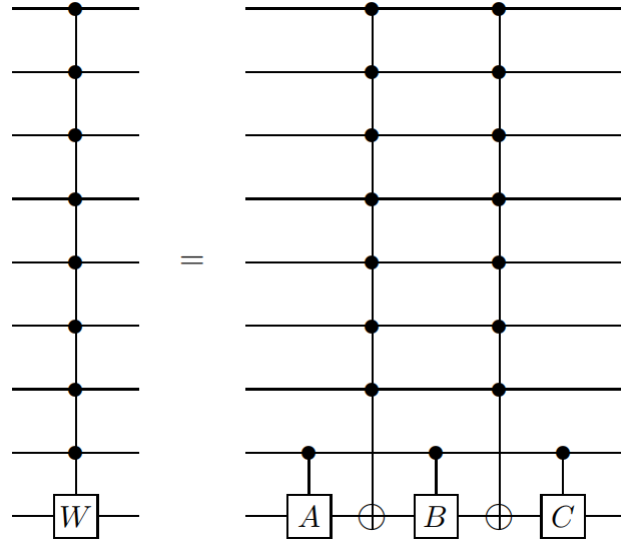
这一部分, 我们介绍针对多控 G 门的综合方案。首先依据 Barenco^[23] 的方案将多控 G 门分解为多控 Toffoli 门, 然后依据 Gidney^[22] 的方案将多控 Toffoli 门分解到 Toffoli 门。此外, 关于 Toffoli 门的一般分解可以参考 Barenco^[23] 的方案, 这里不再赘述。

3.6.1 多控 G 门分解为多控 Toffoli 门

依据 Barenco^[23] 的方案, 单控 G 门可以由两个 CNOT 门和单比特门来实现, 多控 G 门 $C^n G$ 可以由两个 $C^{n-1} NOT$ 门和三个单控 G 门来实现, 具体如图一—图二所示。



图一 控制 G 门的分解示意图^[23]

图二 多控 G 门的分解示意图^[23]

3.7 多控 Toffoli 门的分解

这里针对多控 Toffoli 门的分解可以使用借位辅助比特，由于本文中，只要不是 $n - 1$ 个控制位的 $C^{n-1}NOT$ 门，均可以使用该方法。

针对 $C^n NOT$ 门的分解，只要存在一个辅助比特，就可以进行分解与综合。这里分解综合分为两步，第一步将 $C^n NOT$ 缩减到一般的控制位，第二步将后续多控 Toffoli 门分解到 Toffoli 门。

第一步分解具体而言：

当 n 为奇数时，可以将 $C^{2n-1}NOT$ 门分解为四个 $C^n NOT$ 门：

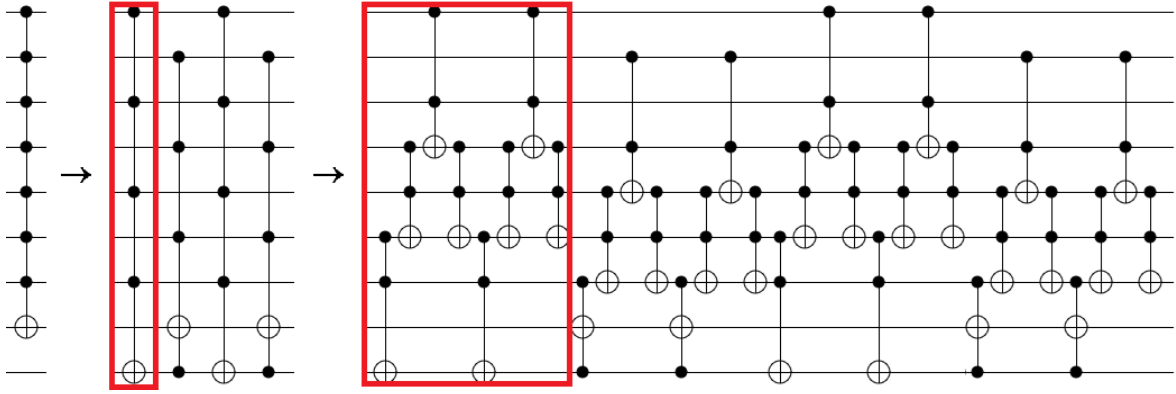
$$C^{2n-1}NOT = 4 \times C^n NOT$$

当 n 为偶数时，可以将 $C^{2n}NOT$ 门分解为两个 $C^n NOT$ 门和两个 $C^{n+1}NOT$ 门：

$$C^{2n}NOT = 2 \times C^n NOT + 2 \times C^{n+1}NOT$$

第二部分解，针对第一步分解后的多控 Toffoli 门进行处理，如图三红框所示。针对这些门而言，如果其有 m 个控制位，那么我们至少拥有 $m - 1$ 个借位辅助比特。然而，其仅需 $m - 2$ 个借位辅助比特就可以分解为 $4(m - 1)$ 个 Toffoli 门：

$$C^m NOT = 4(m - 1) \times C^2 NOT$$

图三 多控 Toffoli 门的分解示意图^[22]

4 数值实验和应用案例

在数值实验这一部分，我们首先在全连接结构上，针对随机态进行多次实验测算我们的算法线路长度，接着在 Heavy-hex, Square 两个特定结构上进行 MAPPING，同样对随机态进行多次实验测算我们的线路长度。此外，我们给出了一个稀疏量子态制备的案例。

4.1 随机态

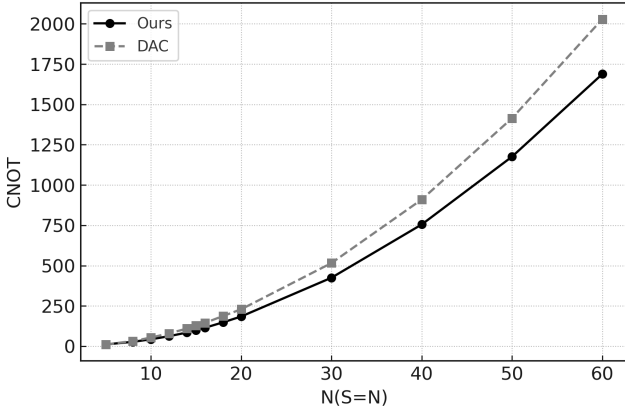
针对本文算法在 *qubits* 数量 N 在 $10 - 50$ ，非零系数的数量 S 在 $N^1 - N^{1.5}$ 的范围内进行数值实验结果在下表 1。

Table 1 量子态稀疏相位数量 S 作为 Qubits 数量 N 的幂函数时，不同 N 值下电路规模数据表，其中线路复杂度系数为 CNOT 门数与 SN 的比值。值得注意的是，当 N 的值较小时，其幂值差异较小，我们这里四舍五入处理

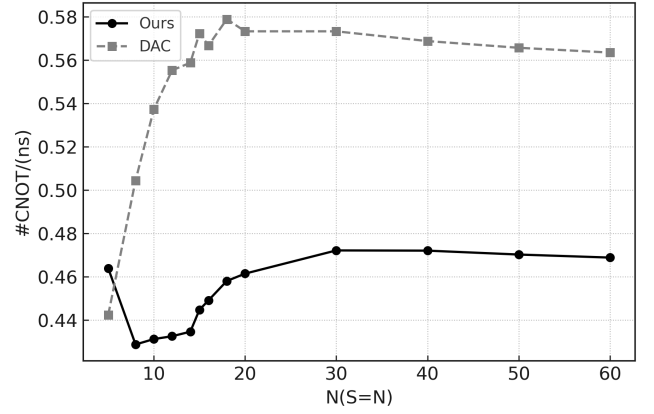
$S = N^k$	<i>qubit</i> = 10		<i>qubit</i> = 20		<i>qubit</i> = 30		<i>qubit</i> = 40		<i>qubit</i> = 50	
	CNOT	系数	CNOT	系数	CNOT	系数	CNOT	系数	CNOT	系数
$S = N^1$	41.13	0.4113	182.60	0.4565	422.98	0.4700	753.36	0.4709	1173.75	0.4695
$S = N^{1.1}$	55.50	0.4408	265.48	0.4919	619.55	0.4899	1140.79	0.4930	1826.11	0.4940
$S = N^{1.2}$	78.32	0.4942	382.43	0.5251	939.05	0.5285	1792.15	0.5356	2973.58	0.5439
$S = N^{1.3}$	111.17	0.5572	557.58	0.5675	1450.87	0.5810	2859.18	0.5908	4833.90	0.5980
$S = N^{1.4}$	154.99	0.6170	834.83	0.6297	2279.77	0.6498	4626.86	0.6612	8066.03	0.6747
$S = N^{1.5}$	216.51	0.6846	1260.93	0.7049	3608.86	0.7321	7623.90	0.7534	13616.88	0.7703

对图表进行分析可知，随着 S 数量增加，系数存在明显的上升。随这个 N 的增加，系数呈现先上升，最后趋于稳定的趋势。Gleinig 与 Hoefler^[18] 在 2021 年 DAC 上提出的相位缩减方法作为当前不使用辅助比特的最优方法，具有重要参考价值。为展示本文算法优势，我们对本文提出的算法与其进行了数值测试与性能比对。

我们均匀随机抽取 k 个长度为 n 的二进制字符串。由于振幅对本文算法与 Gleinig^[18] 提出的算法均不影响电路规模，因此无需对振幅进行采样。需要说明的是，由于 CNOT 门数量通常能有效表征电路规模，故实验中以 CNOT 门数量而非电路规模作为计量指标，具体结果如图四至图六所示。

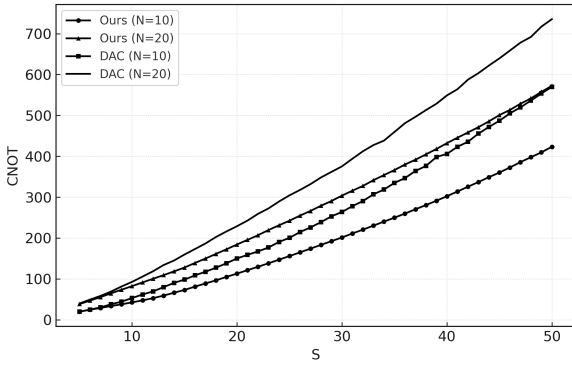


(a) CNOT 门数

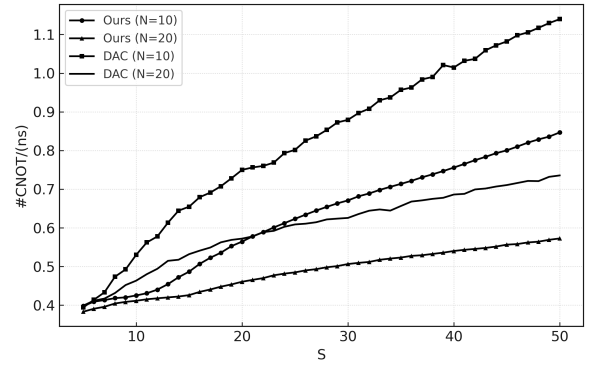


(b) CNOT 门系数

图四 当态稀疏相位数量 S 等于 Qubits 数量 N 时, (a) CNOT 门数与 S, N 的变化关系对比图, (b) 线路复杂度系数与 S, N 的变化关系对比图, 其中线路复杂度系数为 CNOT 门数与 SN 的比值

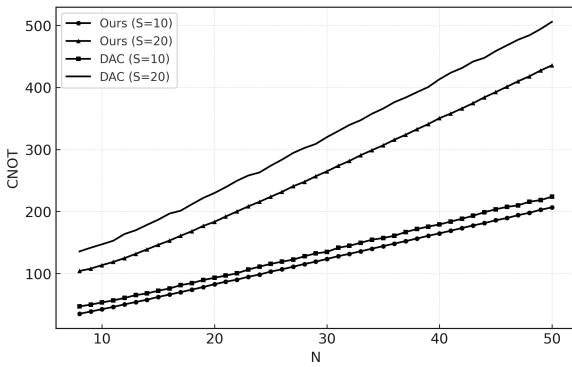


(a) CNOT 门数

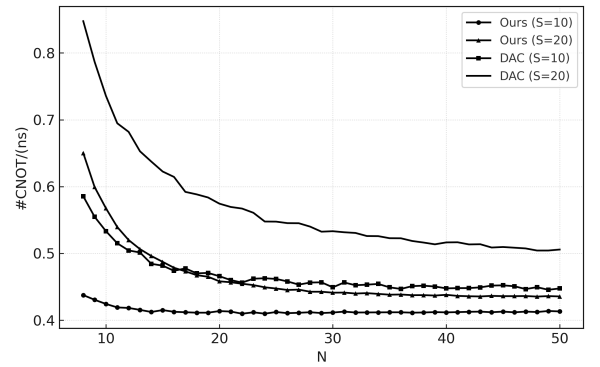


(b) CNOT 门系数

图五 当 Qubits 数量 N 固定时, (a) CNOT 门数与 S 的变化关系对比图, (b) 线路复杂度系数与 S 的变化关系对比图, 其中线路复杂度系数为 CNOT 门数与 SN 的比值



(a) CNOT 门数



(b) CNOT 门系数

图六 当态稀疏相位数量 S 不变时, (a) CNOT 门数与 N 的变化关系对比图, (b) 线路复杂度系数与 N 的变化关系对比图, 其中线路复杂度系数为 CNOT 门数与 SN 的比值

特别的，在比特数量和量子非零相位数量都为 10 时，本文算法相较于 Gleinig^[18] 提出的算法可以节省 19.73% 的 *CNOT* 门；当比特数量和量子非零相位数量都为 15 时，我们算法可以节省 22.28% 的 *CNOT* 门；当比特数量和量子非零相位数量都为 30 时，我们算法可以节省 17.63% 的 *CNOT* 门；当比特数量和量子非零相位数量都为 50 时，我们算法可以节省 16.79% 的 *CNOT* 门。并且随着非零相位数量的增加，我们的算法会表现出更大的优势。当在比特数量为 15 非零相位数量为 20 时，我们算法可以节省 21.19% 的 *CNOT* 门；当在比特数量为 15 非零相位数量为 50 时，我们算法可以节省 20.40% 的 *CNOT* 门；当在比特数量为 15 非零相位数量为 50 时，我们算法可以节省 23.28% 的 *CNOT* 门。

4.2 特定结构的 MAPPING

考虑算法在 NISQ 设备上的适配性，我们将全连接结构的量子线路通过增加线路局部性的方式进行重写与映射：在保持逻辑功能等价的前提下，优先将远距离的两比特相互作用转化为近邻交互，最小化线路长度，进而降低误差、提升在噪声环境下的可执行性。基于这一流程，我们将原始线路分别映射到两类代表性的硬件耦合图，Heavy-Hex 与 Square_25，其结构示意图如图七所示。

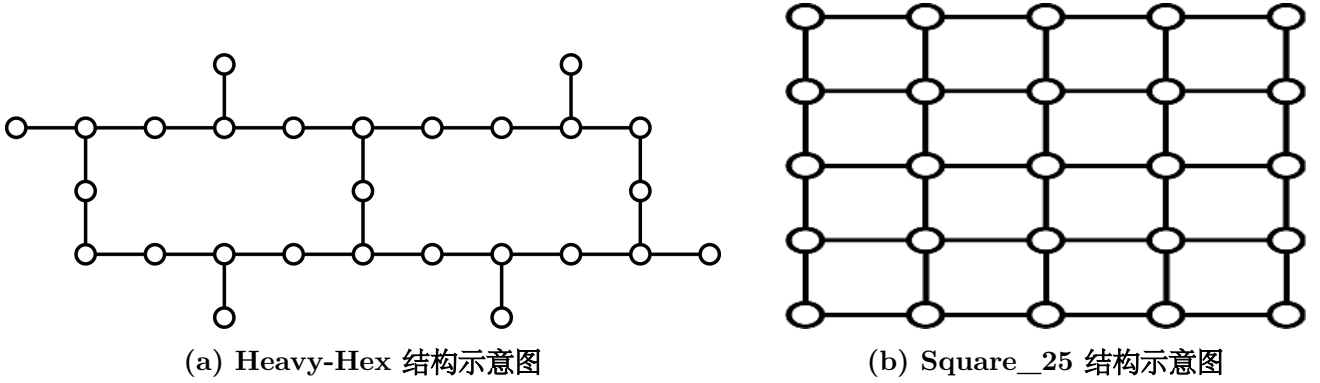


图 七 两种特定结构示意图

表 2和表 3为我们算法在 Heavy-Hex 上映射的线路长度结果。在该拓扑下，我们的方法相较于目前最优方法^[18,24]在量子比特规模为 10 和 20 时均取得显著优势：当 $qubit=10$ 时平均节省 19.70% 的线路长度，当 $qubit=20$ 时平均节省 15.65% 的线路长度；整体节省率随 S 增大呈上升趋势，最高可以节省 32.18% 的线路长度，显示出良好的可扩展性与稳健性。

Table 2 Heavy-Hex 实验结果 ($qubit=10$)

S	5	10	15	20	25	30	35	40	45	50	55	60	65	70	75	80	85	90	95	100
DAC	1.024	1.3327	1.5760	1.6210	1.6883	1.7927	1.9324	2.0742	2.2524	2.4312	2.4646	2.6772	2.7513	2.9855	3.1535	3.3013	3.5546	3.6488	3.7862	3.9236
Ours	1.049	1.3840	1.3393	1.5288	1.5450	1.5690	1.6441	1.6615	1.7908	1.8495	1.9586	2.0004	2.0737	2.1974	2.2417	2.3347	2.4635	2.5174	2.6252	2.6610
Reduce	-0.0244	-0.0385	0.1502	0.0569	0.0849	0.1248	0.1492	0.1990	0.2050	0.2393	0.2053	0.2528	0.2463	0.2640	0.2891	0.2928	0.3070	0.3101	0.3067	0.3218

Table 3 Heavy-Hex 实验结果 ($qubit=20$)

S	10	20	30	40	50	60	70	80	90	100	110	120	130	140	150	160	170	180	190	200
DAC	2.5633	2.8205	2.9272	3.1761	3.2670	3.4259	3.5124	3.6122	3.7440	3.8530	3.9010	4.0526	4.2135	4.3220	4.3926	4.5586	4.5536	4.6348	4.7648	4.8609
Ours	2.2370	2.5027	2.6540	2.6796	2.7785	2.8688	2.9463	3.0694	3.1325	3.2569	3.3362	3.4291	3.4697	3.6185	3.6329	3.7034	3.7564	3.8596	3.9304	3.9738
Reduce	0.1273	0.1127	0.0933	0.1563	0.1495	0.1626	0.1612	0.1503	0.1633	0.1547	0.1448	0.1538	0.1765	0.1628	0.1729	0.1876	0.1751	0.1673	0.1751	0.1825

表 4和表 5为我们算法在 Square_25 上映射的线路长度结果。在该拓扑结构上，我们的优势进一步扩大：当 $qubit=10$ 时平均节省 26.18% 的线路长度，当 $qubit=20$ 时平均节省 20.86% 的线路长度。其中最大可以节省 37.60% ($qubit=10, S=185$)，节省率在不同 S 下波动较小，表现更为稳定。

Table 4 Square-25 实验结果 ($qubit=10$)

S	5	10	15	20	25	30	35	40	45	50	55	60	65	70	75	80	85	90	95	100
DAC	0.656	0.8427	1.0338	1.0700	1.1411	1.2473	1.3964	1.5455	1.6993	1.8807	1.9192	2.1204	2.1993	2.3032	2.5234	2.5923	2.8085	2.9466	3.1161	3.2119
Ours	0.668	0.7887	0.8916	0.9663	1.0293	1.0287	1.0790	1.1263	1.1425	1.2460	1.3304	1.3468	1.4635	1.5212	1.6373	1.7036	1.7526	1.8668	1.9706	2.0236
Reduce	-0.0183	0.0641	0.1376	0.0969	0.0979	0.1753	0.2273	0.2712	0.3276	0.3375	0.3068	0.3649	0.3346	0.3395	0.3511	0.3428	0.3760	0.3664	0.3676	0.3700

Table 5 Square-25 实验结果 (qubit=20)

S	10	20	30	40	50	60	70	80	90	100	110	120	130	140	150	160	170	180	190	200
DAC	1.4493	1.6375	1.7286	1.9060	1.9993	2.1419	2.2244	2.3190	2.4333	2.5237	2.6204	2.7434	2.8234	2.8745	2.9831	3.0661	3.1577	3.2489	3.3139	3.3999
Ours	1.2573	1.4308	1.5193	1.5688	1.6063	1.7104	1.7857	1.8250	1.9108	1.9461	2.0546	2.0947	2.1375	2.2346	2.2775	2.3527	2.4073	2.4533	2.4920	2.5653
Reduce	0.1325	0.1262	0.1210	0.1769	0.1966	0.2015	0.1972	0.2130	0.2147	0.2289	0.2159	0.2365	0.2429	0.2226	0.2365	0.2327	0.2377	0.2449	0.2480	0.2455

总体而言，无论是超导芯片常见的 Heavy-Hex 拓扑，还是规则网络的 Square_25，我们的方法在绝大多数设置下均能稳定且显著地压缩线路长度，并且随着非零相位数量 S 的增大，优势进一步放大。同时，得益于面向拓扑约束的设计，我们的方法对 NISQ 设备具有良好适配性：在针对特定线路结构的非全连接拓扑映射中，所需资源更低。

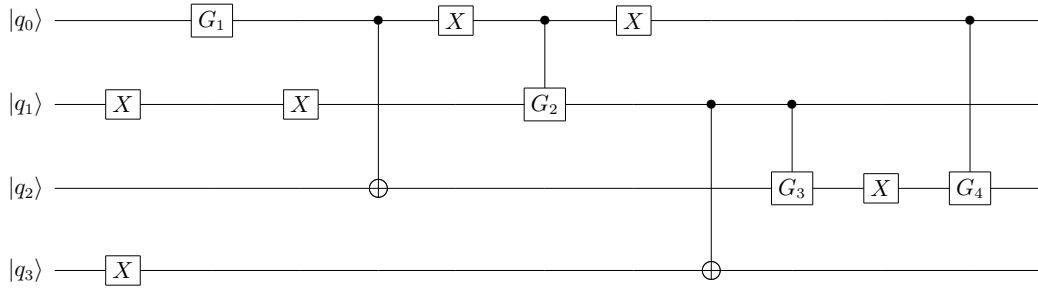
4.3 稀疏量子态制备工具

此外，我们将我们的算法封装成了一个 SQSP 软件包，该工具可以高效的制备稀疏态，具体使用可以参考附录 A。接下来，我们分析该软件包制备下述量子态的过程，并且给出了制备的线路图。

$$|\psi\rangle = \alpha_3 |0011\rangle + \alpha_4 |0100\rangle + \alpha_6 |0110\rangle + \alpha_9 |1001\rangle + \alpha_{11} |1011\rangle$$

其中 $\alpha_x \in \mathbb{C}$ 为复数振幅，满足归一化条件：

$$\sum_x |\alpha_x|^2 = 1.$$



图八 稀疏量子态制备线路示意图

在制备这个稀疏态的量子线路中，我们的线路长度（CNOT 门数量）为 8。在 G_1 门之前，我们完成了 $|0000\rangle$ 到 $\alpha_5 |0101\rangle$ 的制备，在 G_2 之前，我们完成了 $\alpha_3 |0011\rangle + \alpha_9 |1001\rangle$ 的制备；在 G_3 之前，我们完成了 $\alpha_1 |0001\rangle + \alpha_4 |0100\rangle + \alpha_{11} |1011\rangle$ 的制备；在 G_4 之前，我们完成了 $\alpha_3 |0011\rangle + \alpha_4 |0100\rangle + \alpha_6 |0110\rangle + \alpha_9 |1001\rangle$ 的制备；经过 G_4 ，完成了该稀疏态 $\alpha_3 |0011\rangle + \alpha_4 |0100\rangle + \alpha_6 |0110\rangle + \alpha_9 |1001\rangle + \alpha_{11} |1011\rangle$ 的制备。

5 总结及未来展望

我们在本文中提出了一个稀疏量子态制备的自动化设计方法。我们的算法是一种多项式时间算法，实现该算法的经典时间复杂度为 $O(|S|^2 n \log n)$ 。该算法实现的线路长度为 $O(|S|n)$ ，但是凭借其系数上的优异表现，我们的算法在制备稀疏态的线路长度方面具有显著的优势，是目前已知最优。

我们认为，在稀疏态制备的线路长度及线路深度方面的理论界限的研究是具有重要意义的，并且在相对的理论界限下，针对低系数的面向应用的算法更具有重要意义。此外，针对现有量子设备的连接结构的限制，针对该算法的在链接限制的结构上的线路长度优化是值得探究的。由于我们采用的随机态进行的算法数值评估，我们认为针对现有 QAOA、QML 等常用量子算法的初始量子态的制备进行进一步的研究是具有价值的。

参考文献

- [1] DEUTSCH D, JOZSA R. Rapid solution of problems by quantum computation[J]. Proceedings of the Royal Society of London. Series A: Mathematical and Physical Sciences, 1992, 439(1907): 553-558.
- [2] GROVER L K. A fast quantum mechanical algorithm for database search[C]//Proceedings of the twenty-eighth annual ACM symposium on Theory of computing. 1996: 212-219.
- [3] SHOR P W. Polynomial time algorithms for discrete logarithms and factoring on a quantum computer[C]//Algorithmic Number Theory, First International Symposium, ANTS-I, Ithaca, NY, USA, May 6-9, 1994, Proceedings. 1994.
- [4] BIAMONTE J, WITTEK P, PANCOTTI N, et al. Quantum machine learning[J]. Nature, 2017, 549(7671): 195-202.
- [5] BERRY D W, CHILDS A M, CLEVE R, et al. Simulating hamiltonian dynamics with a truncated taylor series[J]. Physical review letters, 2015, 114(9): 090502.
- [6] LOW G H, CHUANG I L. Optimal hamiltonian simulation by quantum signal processing[J]. Physical review letters, 2017, 118(1): 010501.
- [7] G. H. Low and I. L. Chuang. Hamiltonian simulation by qubitization[J]. Quantum, 2019, 3: 163.
- [8] BERRY D W, CHILDS A M, KOTHARI R. Hamiltonian simulation with nearly optimal dependence on all parameters[C]//2015 IEEE 56th Annual Symposium on Foundations of Computer Science. IEEE, 2015: 792-809.
- [9] AARONSON S. Multilinear formulas and skepticism of quantum computing[C]//Proceedings of the thirty-sixth annual ACM symposium on Theory of computing. 2004: 118-127.
- [10] KNILL E. Approximation by quantum circuits[A]. 1995.
- [11] MOTTONEN M, VARTIAINEN J J, BERGHOLM V, et al. Transformation of quantum states using uniformly controlled rotations[A]. 2004.
- [12] BAUSCH J. Fast black-box quantum state preparation[J]. Quantum, 2022, 6: 773.
- [13] GONZALEZ-CONDE J, WATTS T W, RODRIGUEZ-GRASA P, et al. Efficient quantum amplitude encoding of polynomial functions[J]. Quantum, 2024, 8: 1297.
- [14] HOLMES A, MATSUURA A Y. Efficient quantum circuits for accurate state preparation of smooth, differentiable functions[C]//2020 IEEE International Conference on Quantum Computing and Engineering (QCE). IEEE, 2020: 169-179.
- [15] MARIN-SANCHEZ G, GONZALEZ-CONDE J, SANZ M. Quantum algorithms for approximate function loading [J]. Physical Review Research, 2023, 5(3): 033114.
- [16] RATTEW A G, KOCZOR B. Preparing arbitrary continuous functions in quantum registers with logarithmic complexity[A]. 2022.
- [17] SANDERS Y R, LOW G H, SCHERER A, et al. Black-box quantum state preparation without arithmetic[J]. Physical review letters, 2019, 122(2): 020502.
- [18] GLEINIG N, HOEFLER T. An efficient algorithm for sparse quantum state preparation[C]//2021 58th ACM/IEEE Design Automation Conference (DAC). IEEE, 2021: 433-438.
- [19] MALVETTI E, ITEN R, COLBECK R. Quantum circuits for sparse isometries[J]. Quantum, 2021, 5: 412.
- [20] LI L, LUO J. Nearly optimal circuit size for sparse quantum state preparation[A]. 2024.
- [21] MAO R, TIAN G, SUN X. Toward optimal circuit size for sparse quantum state preparation[J]. Physical Review A, 2024, 110(3): 032439.
- [22] GIDNEY C. Constructing large controlled nots[EB/OL]. 2015. <https://algassert.com/circuits/2015/06/05/Constructing-Large-Controlled-Nots.html>.

-
- [23] BARENCO A, BENNETT C H, CLEVE R, et al. Elementary gates for quantum computation[J]. Physical review A, 1995, 52(5): 3457.
- [24] ZOU H, TREINISH M, HARTMAN K, et al. Lightsabre: A lightweight and enhanced sabre algorithm[A]. 2024.

附录 A.SQSP 软件包的使用

为便于复现实验结果及促进本研究所提稀疏量子态制备 (Sparse Quantum State Preparation, SQSP) 方法的应用, 我们公开了相应的软件包。针对软件包的使用, 读者可以产看我们的 Readme.md 文档, 也可以按照以下步骤使用我们的软件进行稀疏态制备。本附录简要介绍该软件包的结构、依赖关系及使用方法, 供读者参考。

B.1 软件包概述 SQSP 软件包实现了本文所提出的稀疏量子态制备算法, 支持用户自定义量子比特数、目标态稀疏度及相位矩阵, 能够生成对应的量子线路并评估其资源开销。软件包同时提供线路可视化与多轮性能测试功能, 便于算法分析与比较。

B.2 目录结构

软件包的主目录包含以下核心文件与子模块:

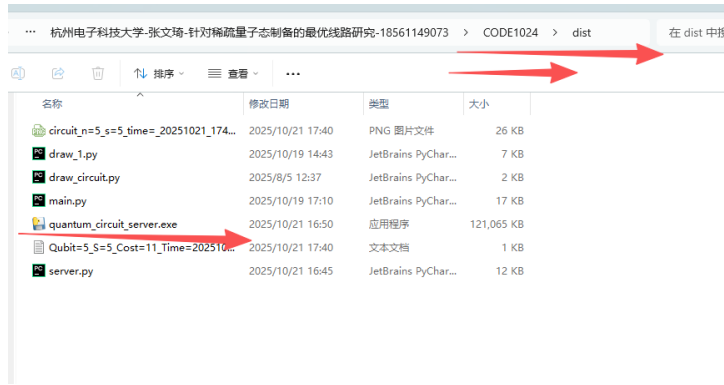
- **quantum_circuit_server.exe**: 主文件, 用于运行 SQSP 算法;
- 针对稀疏量子态制备的最优线路研究.pdf: 算法原理与实现细节的技术文档。

B.3 运行环境与依赖

在运行前, 请确保已正确安装 Python 环境, 并在项目根目录下安装所需依赖。

B.4 基本使用方法

1. 打开软件包



2. 输入量子比特数量与非零相位数量

3. 随后选择量子态输入方式: 随机生成/手动输入/JSON 文件导入

4. 点击运行算法, 程序会检查输入参数的正确性, 算法运行完成后会在同目录下生成对应的量子线路图



B.5 性能测试

为评估算法在不同参数下的平均性能, 可运行多轮测试脚本。在项目根目录下执行: `python main_cost_test.py`。该脚本将自动进行多次实验, 统计线路开销, 输出性能分析结果, 适用于算法对比与鲁棒性验证。

B.6 线路图展示

程序运行完成后会在主目录下生成线路图片文件。示例如下

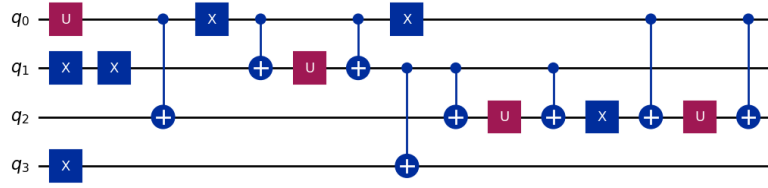


图 九 circuit_n=4_s=5

B.7 线路输出格式

程序运行完成后会在主目录下生成线路文字文件，方便用户调用。示例如下：

```
x 3
x 1
CCCG 0 []
x 1
cx 2 0
x 0
CCCG 1 [0]
x 0
cx 3 1
CCCG 2 [1]
x 2
CCCG 2 [0]
```

注：x为not门，格式为[x 作用比特位]

cx为Cnot门，格式为[x 受控位 控制位]

CCCG为多控G门，格式为[CCCG 受控位 [多控制位]]